GOVERNMENT COLLEGE OF ENGINEERING
DHARMAPURI – 636 704

(Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

NAME :.....

REG.NO :.....

DEGREE & BRANCH : B.E&ECE.....

SUBJECT CODE&TITLE: NM1133& EmbeddedC&MicroController Programming

YEAR & SEM : IV&VII.....

Certified that this Bonafide Record of Work done by the above student of the

..... *Laboratory*

during the year 20 - 20 .

Faculty In-Charge **Head of the Department**

Submitted for the University Practical Examination held on..... at Government College of Engineering, Dharmapuri.

Internal Examiner **External Examiner**

CONTENTS

Exp. No.	Date	Name of Experiment	Page No.	Marks	Signature
1.		Develop a Real-Time Soil Monitoring and Automated Irrigation Control System	6		
2.		Develop a Smart Home Automation for Lighting, Temperature, and Appliances	9		
3.		Develop a Industrial Machine Health Monitoring and Predictive Maintenance System	12		
4.		Develop a Wearable Vital-Sign and Sleep-Quality Monitoring System	15		
5.		Develop a Smart City Traffic, Waste, and Pollution Monitoring system	18		
6.		Develop a Environmental Monitoring System: Pollution, Wildfire, and Wildlife Observation	21		
7.		Develop a Sensor-Based Failure Detection and Maintenance Scheduling system	24		
8.		Develop a Smart Retail Inventory and Customer-Experience Monitoring	27		
9.		Develop a Smart Logistics: Live Tracking and Shipment Security	30		
10.		Develop a Connected Vehicle Performance and Remote Diagnostics System	33		

Exp. No.	Date	Name of Experiment	Page No.	Marks	Signature
11.		Develop a Smart Building Facility Management and Access Control system	36		
12.		Develop a Smart Grid Monitoring and Renewable-Energy Integration system	39		
13.		Develop a Precision Agriculture system: Resource Use, Crop Health, and Yield Support	42		
14.		Develop a Smart Livestock Monitoring, Location Tracking, and Theft Detection system	45		
15.		Develop a Remote Property Monitoring and Smart Security system	48		
16.		Develop a Smart Water Quality Monitoring system	51		
17.		Develop a Smart Parking Management and Payment-System Demonstrator	54		
18.		Develop a Construction-Site Worker Safety and Operations Monitoring System	57		
19.		Develop a Smart Waste Management and Sustainable Recycling System	60		

List of Experiments

1) Develop a Real-Time Soil Monitoring and Automated Irrigation Control System.
2) Develop a Smart Grid Monitoring and Renewable-Energy Integration system.
3) Develop a Industrial Machine Health Monitoring and Predictive Maintenance System.
4) Develop a Wearable Vital-Sign and Sleep-Quality Monitoring System.
5) Develop a Smart City Traffic, Waste, and Pollution Monitoring system.
6) Environmental Monitoring System: Pollution, Wildfire, and Wildlife Observation.
7) Develop a Sensor-Based Failure Detection and Maintenance Scheduling system.
8) Develop a Smart Retail Inventory and Customer-Experience Monitoring.
9) Develop a Smart Logistics: Live Tracking and Shipment Security.
10) Develop a Connected Vehicle Performance and Remote Diagnostics System.
11) Develop a Smart Building Facility Management and Access Control system.
12) Develop a Smart Grid Monitoring and Renewable-Energy Integration system.
13) Develop a Precision Agriculture system: Resource Use, Crop Health, and Yield Support.
14) Develop a Smart Livestock Monitoring, Location Tracking, and Theft Detection system.
15) Develop a Remote Property Monitoring and Smart Security system.
16) Develop a Smart Water Quality Monitoring system.
17) Develop a Smart Parking Management and Payment-System Demonstrator.
18) Develop a Construction-Site Worker Safety and Operations Monitoring System.
19) Develop a Smart Waste Management and Sustainable Recycling System.

1. Introduction and Design Method

Embedded C and Microcontroller Programming is the process of developing software to control hardware devices using microcontrollers. Embedded C is a programming language commonly used to write programs that interact with input devices, sensors, actuators, displays, and communication modules. A typical embedded system consists of a microcontroller, input devices, output devices, a power supply and a software program that processes data and controls the connected hardware.

The design method involves identifying the system requirements, selecting a suitable microcontroller, designing the circuit connections, writing and compiling the Embedded C program, uploading the program to the controller, and testing the system through simulation or hardware implementation. The microcontroller reads input signals, processes them according to the programmed instructions, and generates the required output. This approach is widely used in automation, robotics, IoT applications, industrial control, and smart electronic devices.

1.1 Arduino Uno and ESP32

Feature	Arduino Uno (typical R3)	ESP32 development board (typical)
Logic / GPIO	5 V logic on classic ATmega328P Uno	3.3 V logic; GPIO is generally not 5 V tolerant
Analog input	10-bit ADC; A0–A5 available	ADC capability/pins vary by chip and board; many use 12-bit readings
Connectivity	No built-in Wi-Fi/Bluetooth on classic Uno	Wi-Fi and Bluetooth available on many ESP32 variants
Typical use	Simple local sensing and actuation	Connected nodes, more complex sensing, remote dashboards
Important check	Use correct board pin labels and library versions	ADC range, pin strapping, USB/serial pin conflicts vary by board

1.2 General workflow

1. Define the physical quantity and intended decision.
2. Select a sensor with a suitable range, interface, and accuracy.
3. Check supply voltage, signal levels, current, grounding, and environmental protection.
4. Wire and test the sensor alone before adding actuators.
5. Calibrate with known reference conditions or values.
6. Implement filtering, threshold logic, invalid-data handling, and logging.
7. Test normal, boundary, fault, and recovery conditions.
8. Record measured results, uncertainty, and deviations from expected behavior.

1.3 General wiring and code cautions

- Never drive motors, pumps, relays, solenoids, or buzzers beyond GPIO ratings; use a suitable driver.
- A relay module may be active-LOW or active-HIGH. Confirm its behavior before connecting a load.
- For ESP32 boards, do not feed 5 V into GPIO or ADC pins unless the board documentation explicitly supports it.

EXP NO :1 Real-Time Soil Monitoring and Automated Irrigation Control
DATE:

Application domain: Smart agriculture

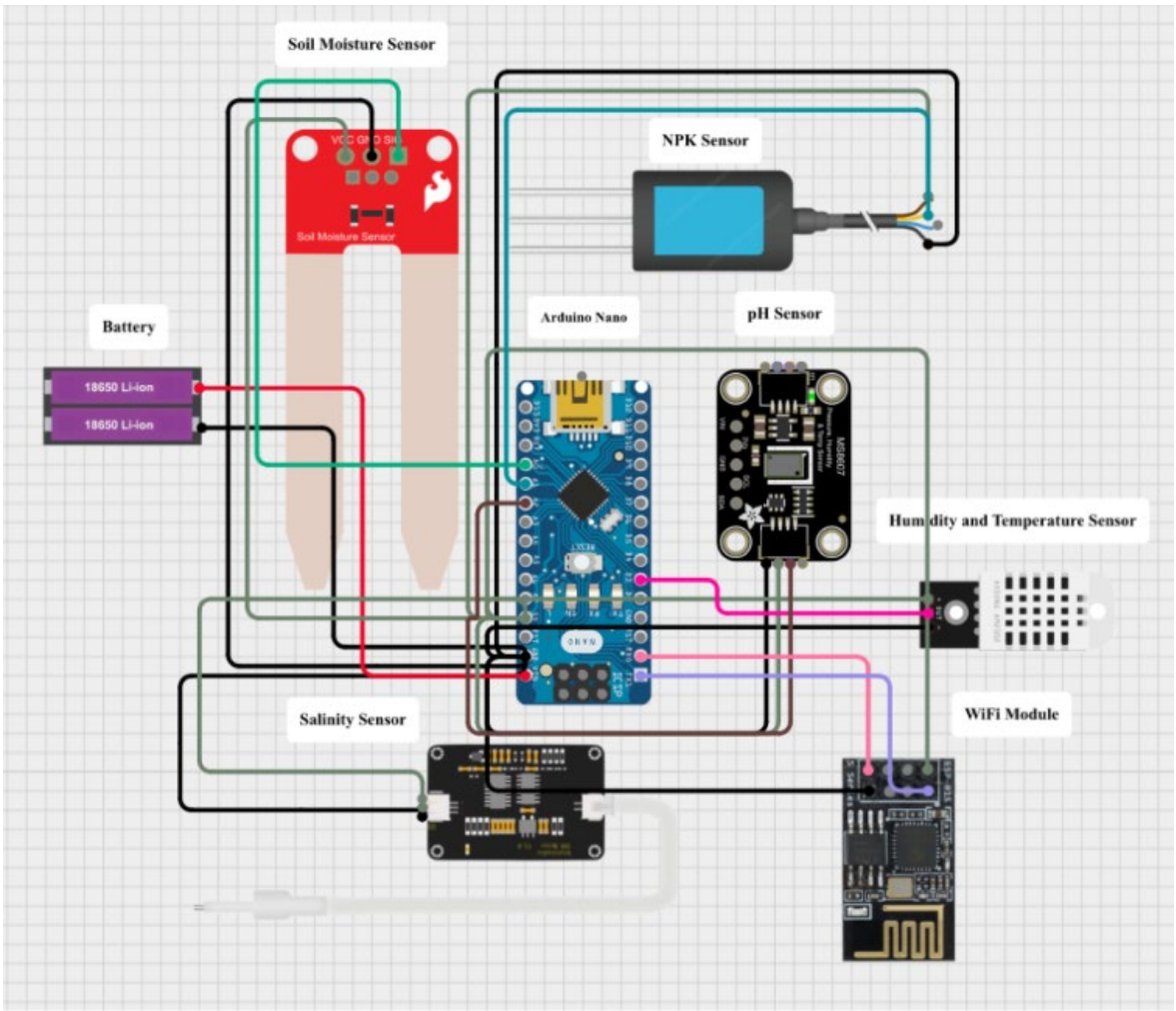


Figure 1 : Smart Agriculture: illustrative system block diagram and project visual.

Abstract

Measures soil moisture and optionally temperature, then switches a water pump when the soil is dry. The design separates low-voltage sensing/control from the pump power circuit.

Objectives

- Measure soil moisture at regular intervals.
- Set an adjustable moisture threshold for irrigation.
- Prevent rapid relay switching with hysteresis and a minimum pump run interval.
- Provide a clear status output for dry soil, watering, and adequate moisture.

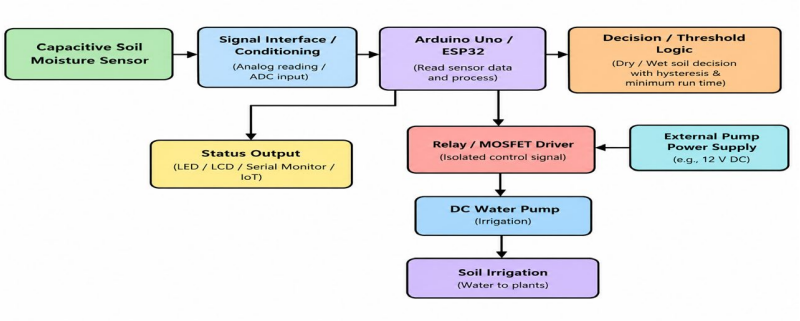
Components Required

Arduino Uno or ESP32; capacitive soil-moisture sensor; 5 V relay module or logic-level MOSFET driver; suitable DC pump; external pump supply; flyback protection if using a bare inductive driver; jumper wires; optional DHT22 and LCD.

Interface and Pin Connections

Moisture sensor AO → A0 (Uno) or an ADC-capable ESP32 pin such as GPIO34; sensor VCC → voltage supported by its module; sensor GND → controller GND. Driver input → D7 (Uno) or GPIO26 (ESP32). Pump uses a separate supply through a properly rated driver/relay; never power the pump from a GPIO pin.

Block Diagram



Working Principle

Capacitive probes estimate water content from changes in dielectric properties and are generally more corrosion-resistant than exposed resistive probes. Calibrate readings in dry and wet soil. A threshold plus hysteresis avoids relay chatter.

Implementation Procedure

Record the sensor reading in dry soil and in well-watered soil; choose a threshold from these measurements.

1. Connect the sensor and driver with the pump supply disconnected.
2. Upload the code and verify status messages using Serial Monitor.
3. Test the driver with a low-voltage lamp or meter before connecting the pump.
4. Check for leaks, dry-run risk, and safe electrical separation before field use.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// Arduino Uno example: capacitive sensor + relay module
const int MOISTURE_PIN = A0;
const int RELAY_PIN = 7;
const int DRY_THRESHOLD = 650; // calibrate for your sensor
const int WET_THRESHOLD = 580; // hysteresis; calibrate
bool watering = false;

void setup() {
  Serial.begin(9600);
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, HIGH); // assumes active-LOW relay; verify module
}
void loop() {
```

```
  int raw = analogRead(MOISTURE_PIN);
  Serial.print("Moisture raw: "); Serial.println(raw);
  if (!watering && raw >= DRY_THRESHOLD) watering = true;
  if (watering && raw <= WET_THRESHOLD) watering = false;
  digitalWrite(RELAY_PIN, watering ? LOW : HIGH);
  Serial.println(watering ? "IRRIGATION REQUESTED" : "SOIL OK");
  delay(1000);
      .
}
```

Proposed Test Plan

Test condition	Expected observation
Dry soil	Reading crosses the dry threshold; irrigation output is requested.
Moist soil	Pump output remains off.
Sensor disconnected	Reading is treated as invalid where possible; system should fail safely.
Repeated threshold crossing	Hysteresis/minimum run timing prevents rapid switching.

Expected Results

In bench tests, readings should change between dry and wet soil and the controller should switch the output at the chosen threshold. These are expected outcomes, not experimentally verified results.

Limitations, Safety, and Future Scope

Sensor readings depend on soil type, salinity, probe placement, and supply voltage. Use a pump driver rated for the motor's startup current, add a water-level interlock, and avoid leaving a pump unattended.

Future scope: Add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.

EXP NO:2 Smart Home Automation for Lighting, Temperature, and Appliances

DATE:

Application domain: Smart buildings

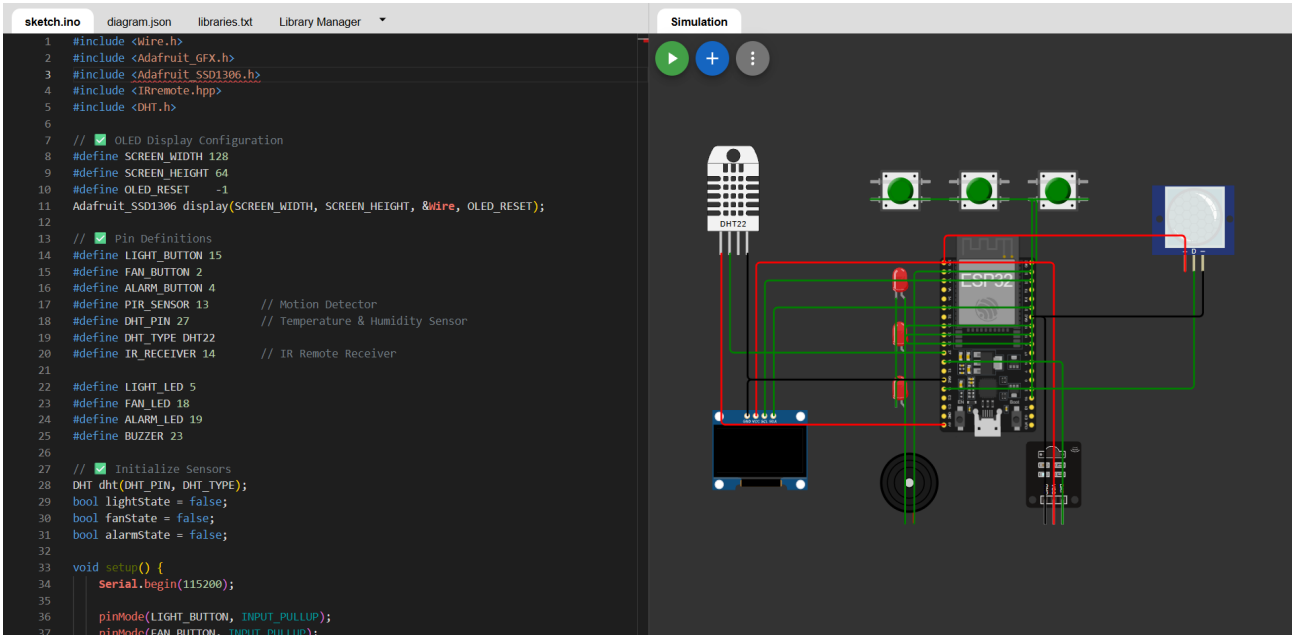


Figure 2. Smart Home Automation for Lighting, Temperature, and Appliances

Abstract

Combines occupancy/light and temperature sensing with controlled lighting and a low-voltage appliance demonstrator. Real household mains loads require an enclosed, certified switching device and qualified installation.

Objectives

- Measure ambient light and room temperature.
- Switch a demonstration light based on light level or occupancy.
- Apply a temperature threshold to a fan demonstrator.
- Expose configurable thresholds for routine-based automation.

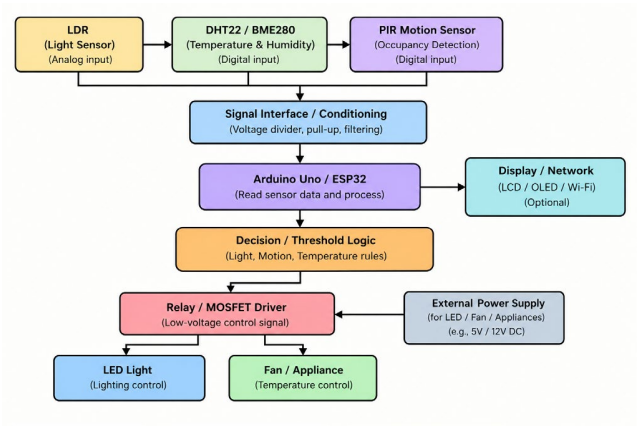
Components Required

Arduino Uno or ESP32; LDR with 10 kΩ resistor or BH1750; DHT22/BME280; PIR motion sensor; relay module or MOSFET driver; LED/fan demonstrator; display or Wi-Fi dashboard optional.

Interface and Pin Connections

LDR divider midpoint → A0 (Uno) or ADC pin (ESP32); DHT22 data → D2 (Uno) or GPIO4 (ESP32), with pull-up if required; PIR OUT → D3/GPIO27; low-voltage driver input → D8/GPIO26. Share controller ground for low-voltage modules.

Block Diagram



Working Principle

The LDR voltage divider converts light level into an analog value. A temperature sensor provides environmental data, while a PIR detects changes in infrared energy. Rules combine these signals to make automation decisions.

Implementation Procedure

1.Calibrate the LDR value in bright and dim conditions.
2.Read temperature and confirm plausible values in Serial Monitor.
3.Test PIR detection without connecting any mains appliance.
4.Set thresholds and verify the rule logic for occupied/unoccupied conditions.
5.For real mains loads, use a certified enclosed smart switch and follow local electrical codes.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// Arduino Uno: LDR + DHT22 + PIR + low-voltage outputs
#include <DHT.h>
#define DHTPIN 2
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);
const int LDR_PIN=A0, PIR_PIN=3, LIGHT_OUT=8, FAN_OUT=9;
const int DARK_LEVEL=600; // calibrate
const float TEMP_LIMIT=30.0;

void setup() {
  Serial.begin(9600); dht.begin();
  pinMode(PIR_PIN, INPUT);
  pinMode(LIGHT_OUT, OUTPUT); pinMode(FAN_OUT, OUTPUT);
}
void loop() {
  int light = analogRead(LDR_PIN);
  int motion = digitalRead(PIR_PIN);
  float temp = dht.readTemperature();
  bool valid = !isnan(temp);
  digitalWrite(LIGHT_OUT, (light > DARK_LEVEL && motion) ? HIGH : LOW);
```

```
  digitalWrite(FAN_OUT, (valid && temp >= TEMP_LIMIT) ? HIGH : LOW);
  Serial.print("LDR="); Serial.print(light);
  Serial.print(" motion="); Serial.print(motion);
  Serial.print(" tempC="); Serial.println(valid ? temp : -999);
  delay(1000);

}
```

Proposed Test Plan

Test condition	Expected observation
Bright room	Light output follows the configured bright-light rule.
Dark + motion	Demonstration light turns on when the rule conditions .
Temperature above threshold	Fan output is requested.
No motion timeout	Lighting returns to its configured idle state.

Expected Results

The system should react to light, motion, and temperature thresholds once sensors are calibrated. No household energy savings are claimed until measured over a defined period.

Limitations, Safety, and Future Scope

PIR sensors do not reliably detect a stationary person. Sensor placement, HVAC airflow, and LDR orientation affect results. Never expose mains wiring on a breadboard.

Future scope: Add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.

EXP NO:3 Industrial Machine Health Monitoring and Predictive Maintenance

DATE:

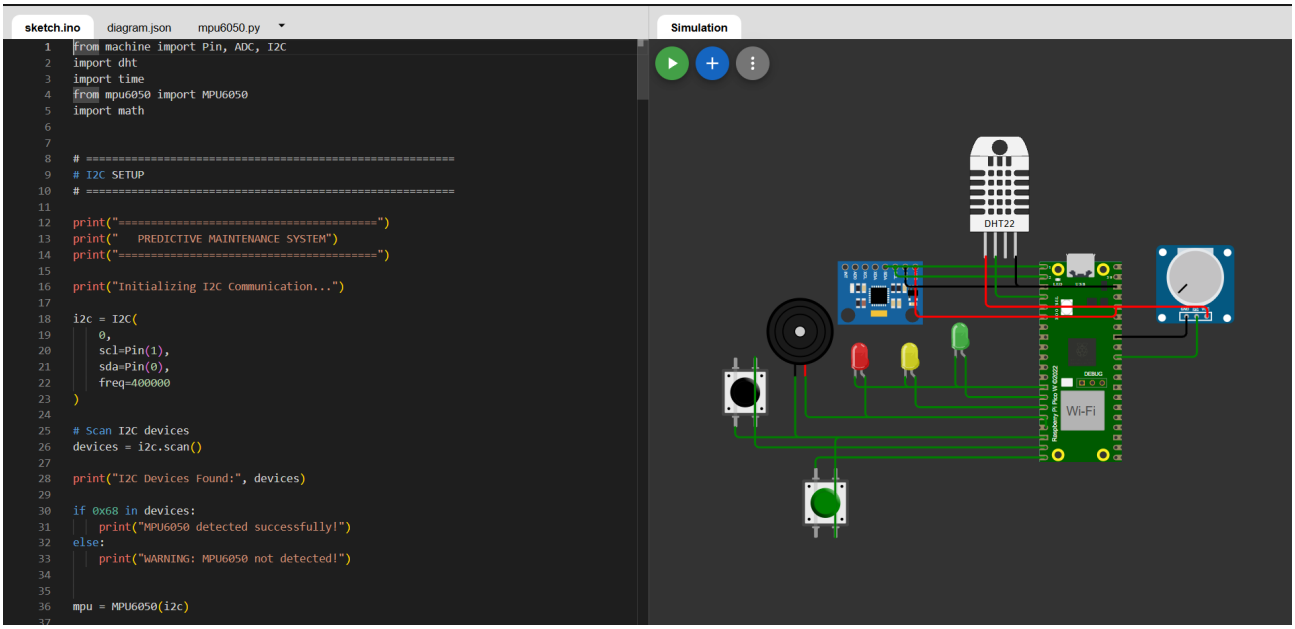


Figure 3. Industrial Machine Health Monitoring: illustrative system block diagram and project visual.

Application domain: Industrial IoT

Abstract

Demonstrates condition monitoring by tracking vibration and temperature. It records threshold events that can indicate imbalance, looseness, overheating, or changing operating conditions.

Objectives

- Acquire vibration and temperature measurements.
- Establish a baseline during normal operation.
- Flag measurements outside configured limits.
- Demonstrate a maintenance alert without directly controlling industrial machinery.

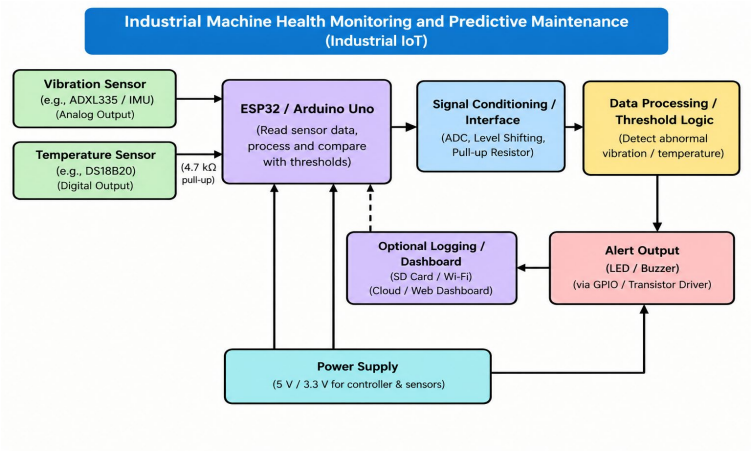
Components Required

ESP32 or Arduino Uno; vibration sensor (analog accelerometer such as ADXL335 or digital IMU); DS18B20/thermistor; status LED/buzzer; optional SD card or Wi-Fi dashboard.

Interface and Pin Connections

Analog vibration output → ADC pin; DS18B20 data → GPIO4/D2 with 4.7 kΩ pull-up to the sensor logic supply; LED → resistor → GPIO; buzzer through a transistor driver if current exceeds GPIO limits.

Block Diagram



Working Principle

Vibration amplitude and temperature trends can reveal changes in machine condition. A simple threshold is suitable for a teaching demonstrator; robust predictive maintenance requires sampling, frequency analysis, historical data, and machine-specific baselines.

Implementation Procedure

1.Mount the sensor firmly to a safe, non-rotating surface.
2.Collect a baseline under normal operation; do not touch moving machinery.
3.Set alert thresholds from machine documentation and baseline data.
4.Run the monitor and log values at consistent intervals.
5.Compare trend records and validate any alert with an appropriate inspection.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// ESP32 example: analog vibration + DS18B20
#include <OneWire.h>
#include <DallasTemperature.h>
const int VIB_PIN=34, LED_PIN=2;
OneWire oneWire(4);
DallasTemperature tempBus(&oneWire);
const int VIB_LIMIT=2200; // determine from baseline

void setup() {
  Serial.begin(115200); pinMode(LED_PIN, OUTPUT);
  tempBus.begin();
}
void loop() {
  int vib=analogRead(VIB_PIN);
  tempBus.requestTemperatures();
  float t=tempBus.getTempCByIndex(0);
  bool alert=(vib>VIB_LIMIT || t>65.0);
  digitalWrite(LED_PIN, alert ? HIGH : LOW);
  Serial.printf("vibration=%d temperature=%.2f C alert=%s\n",
```

```
  vib,t,alert?"YES":"no");
  delay(500);
}
```

Proposed Test Plan

Test condition	Expected observation
Normal idle	Readings remain near the established baseline.
Controlled vibration stimulus	Vibration indicator responds to a safe tap or test fixture.
Warm sensor stimulus	Temperature rises gradually when safely warmed.
Threshold event	Alert activates and is recorded.

Expected Results

Sensor values should be displayed and configured threshold crossings should trigger an alert. Predictive accuracy is not established by a single threshold demo.

Limitations, Safety, and Future Scope

Do not attach sensors to exposed rotating parts. Thresholds vary by machine, sensor mounting, sampling rate, and vibration frequency. This prototype is not a safety-rated shutdown system.

Future scope: Add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.

EXP NO:4 Wearable Vital-Sign and Sleep-Quality Monitoring
DATE:

Application domain: Wearable health technology

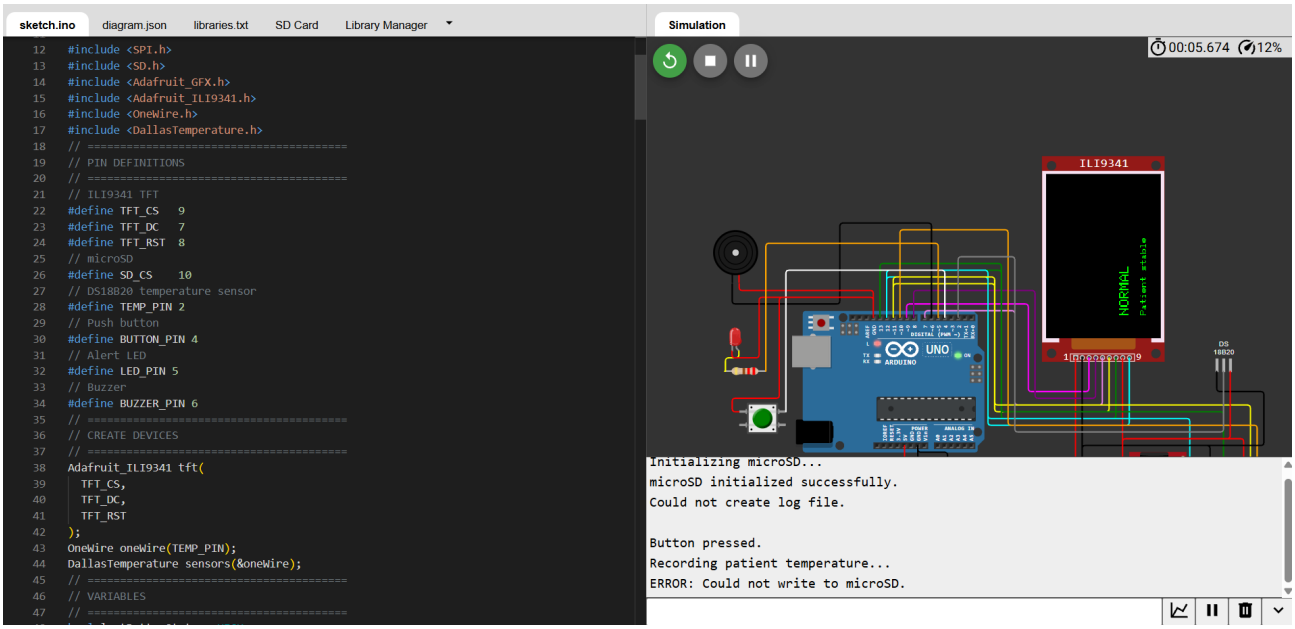


Figure 4:Wearable vital-sign and sleep-quality monitoring

Abstract

A learning prototype records pulse-related optical readings and movement to explore activity and sleep patterns. It is not a medical device and must not diagnose or rule out health conditions.

Objectives

- Read a pulse-oximeter sensor module or heart-rate sensor.
- Track movement using an accelerometer.
- Log time-stamped readings for later review.
- Explain the difference between wellness data and clinical diagnosis.

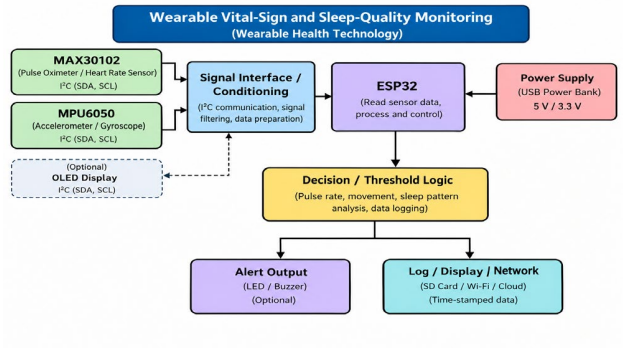
Components Required

ESP32; MAX30102 module; MPU6050 accelerometer/gyroscope; OLED optional; USB power bank; enclosure and strap. Sensor compatibility and libraries must be checked for the exact module.

Interface and Pin Connections

MAX30102 and MPU6050 commonly use I²C: SDA/SCL to the board's I²C pins (Uno A4/A5; ESP32 commonly GPIO21/22). Use 3.3 V-compatible modules and confirm breakout-board voltage support.

Block Diagram



Working Principle

Optical PPG sensors estimate pulse-related changes in blood-volume absorption; accelerometers measure motion. Sleep-stage determination requires validated algorithms and usually more signals than movement alone.

Implementation Procedure

1.Confirm I²C addresses and read sensor samples with a known library example.
2.Keep the optical sensor stable against the skin and avoid excessive pressure.
3.Record pulse-related data and motion alongside timestamps.
4.Compare activity patterns over a night without labeling them as clinical sleep stages.
5.Protect personal data and obtain consent before collecting anyone else's readings.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// ESP32 I2C bring-up template for MAX30102 + MPU6050
// Install SparkFun MAX3010x and Adafruit MPU6050 libraries.
// This template checks the I2C bus; add library-specific acquisition
// after confirming the exact breakout and library versions.
#include <Wire.h>
const int SDA_PIN=21, SCL_PIN=22;
void setup() {
  Serial.begin(115200);
  Wire.begin(SDA_PIN,SCL_PIN);
  Serial.println("I2C scan starting");
  for (byte a=1; a<127; a++) {
    Wire.beginTransmission(a);
    if (Wire.endTransmission()==0) {
      Serial.print("I2C device at 0x");
      Serial.println(a,HEX);
    }
  }
}
void loop() { delay(1000); }
```

Proposed Test Plan

Test condition	Expected observation
Sensor absent	Software reports a sensor error rather than a plausible reading.
Still hand	Motion readings are low and stable.
Gentle movement	Accelerometer values change.
Long recording	Data includes timestamps and missing samples are identified.

Expected Results

The prototype should acquire sensor signals when fit and lighting conditions are suitable. Pulse accuracy and sleep quality must be validated against an appropriate reference before any performance claim.

Limitations, Safety, and Future Scope

Motion, skin tone, perfusion, fit, ambient light, and sensor quality affect readings. Do not use this device for emergency decisions or medical advice.

Future scope: Add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.

EXP NO:5 Smart City Traffic, Waste, and Pollution Monitoring
DATE:

Application domain: Smart city

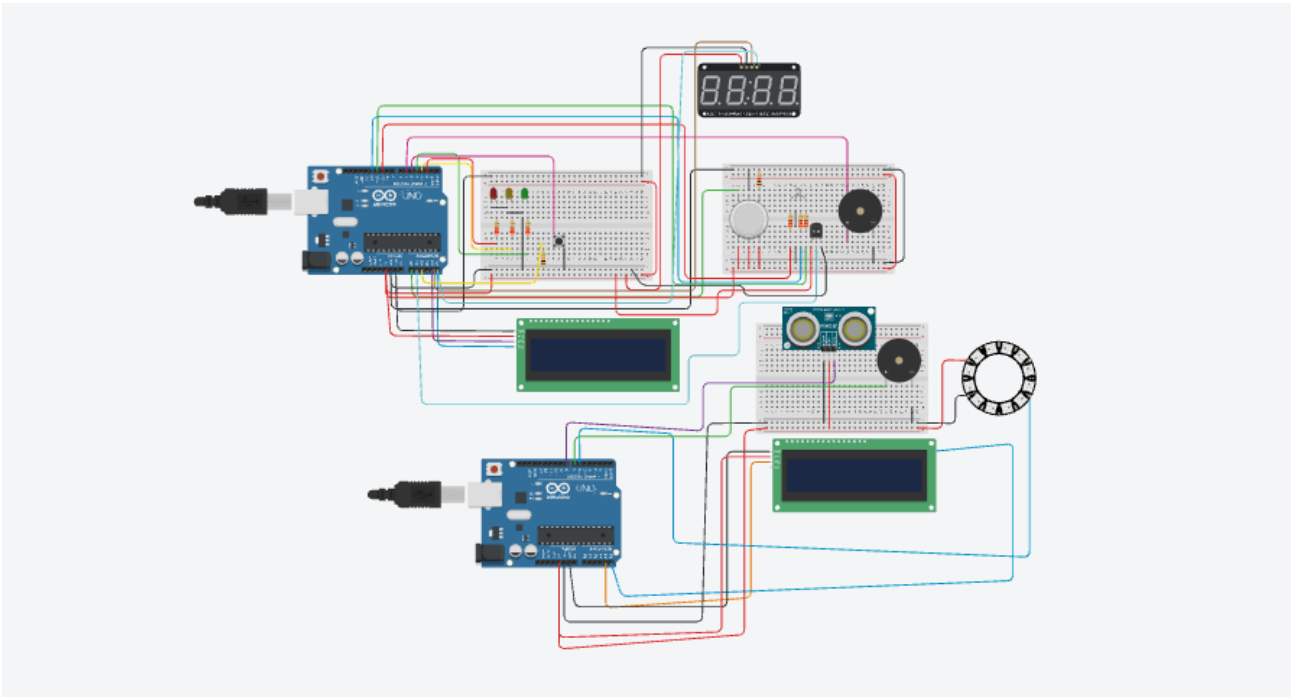


Figure 5:Smart city traffic,waste,and pollution monitoring

Abstract

Combines a traffic counter, fill-level sensor, and environmental sensor readings into a small monitoring node. Each subsystem can be deployed independently before being integrated.

Objectives

- Count vehicle-like events using a break-beam or IR sensor.
- Estimate bin fill level using an ultrasonic sensor.
- Collect air-quality proxy readings from a supported gas sensor.
- Present threshold alerts for monitoring and planning.

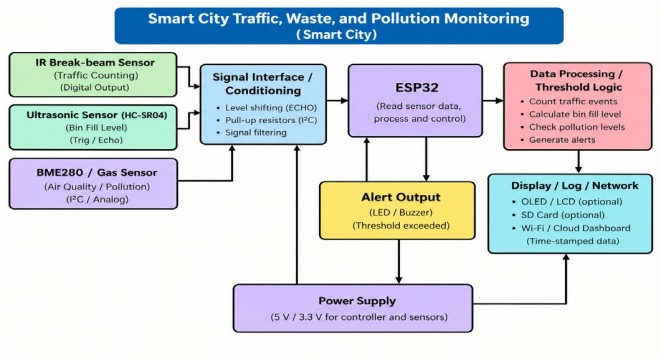
Components Required

ESP32; IR break-beam or reflective sensor; HC-SR04 ultrasonic sensor; BME280 and/or calibrated gas sensor module; LEDs; optional Wi-Fi dashboard. Gas sensors require warm-up and calibration.

Interface and Pin Connections

IR digital OUT → GPIO27; ultrasonic TRIG/ECHO → GPIO5/GPIO18 (use a divider/level shifter on ECHO for ESP32); I²C sensors → GPIO21/22. Never apply 5 V logic directly to ESP32 GPIO.

Block Diagram



Working Principle

Event counting approximates traffic flow at a single point; ultrasonic distance estimates remaining bin space; gas sensors provide indicative readings rather than regulatory air-quality measurements.

Implementation Procedure

1.Test each sensor separately and document its measurement range.
2.Measure the empty-bin and full-bin distances and set limits.
3.Debounce traffic pulses to avoid counting one vehicle multiple times.
4.Publish timestamped data and clearly label sensor estimates.
5.Compare pollution readings with a calibrated reference if they will inform decisions.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// ESP32: ultrasonic bin-level estimate + IR event count
const int TRIG=5, ECHO=18, IR_PIN=27, LED=2;
volatile unsigned long events=0;
unsigned long lastCount=0;
void IRAM_ATTR onEvent() {
  unsigned long now=millis();
  if (now-lastCount>250) { events++; lastCount=now; }
}
float distanceCm() {
  digitalWrite(TRIG,LOW); delayMicroseconds(2);
  digitalWrite(TRIG,HIGH); delayMicroseconds(10);
  digitalWrite(TRIG,LOW);
  long us=pulseIn(ECHO,HIGH,30000);
  if (!us) return -1;
  return us*0.0343/2.0;
}
void setup() {
  Serial.begin(115200);
  pinMode(TRIG,OUTPUT); pinMode(ECHO,INPUT); pinMode(LED,OUTPUT);
  pinMode(IR_PIN,INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(IR_PIN),onEvent,FALLING);
}
void loop() {
```

```
  float d=distanceCm();
  Serial.printf("events=%lu bin_distance_cm=%.1f\n",events,d);
  digitalWrite(LED,(d>0 && d<10)?HIGH:LOW); // near-full alert example
  delay(1000);
}
```

Proposed Test Plan

Test condition	Expected observation
IR beam interruption	Counter increments once per debounced event.
Empty bin	Distance corresponds to a low fill estimate.
Obstacle near sensor	Distance decreases and fill estimate increases.
Gas sensor warm-up	Readings stabilize according to the module datasheet.

Expected Results

The demonstrator should produce traffic-event counts, bin-distance readings, and environmental sensor values. City-scale accuracy is not established by this prototype.

Limitations, Safety, and Future Scope

Traffic events are not necessarily unique vehicles; ultrasonic readings are affected by angled or soft surfaces. Low-cost gas sensors are cross-sensitive and should not be presented as certified pollution measurements.

Future scope: Add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.

EXP NO:6 Environmental Monitoring: Pollution, Wildfire, and Wildlife Observation

DATE:

Application domain: Environmental sensing

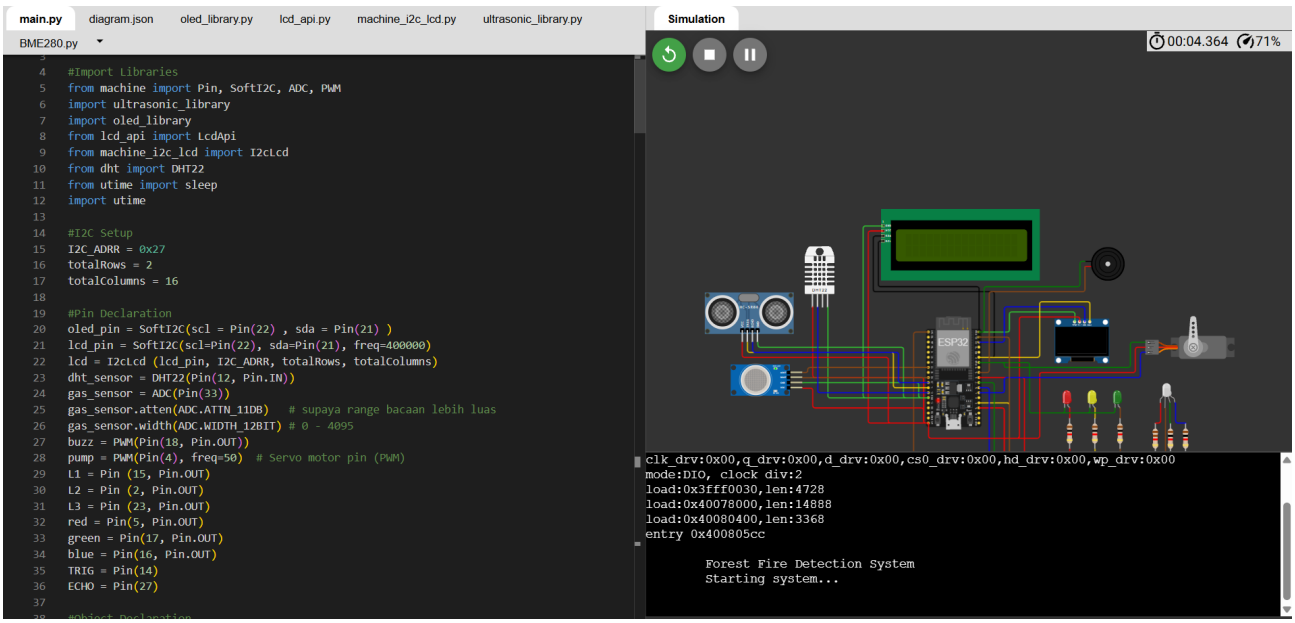


Figure 6:Environmental monitoring:Pollution,Wildfire,And Wildlife Observation

Abstract

Uses temperature, humidity, smoke/gas proxy, and motion sensing to demonstrate environmental event detection. The design is a learning prototype, not a certified wildfire alarm or wildlife-survey instrument.

Objectives

- Record temperature and humidity.
- Detect a configurable rapid temperature rise or sensor threshold.
- Use a PIR or camera-trigger input to record wildlife-like motion events.
- Provide local status and timestamped logging.

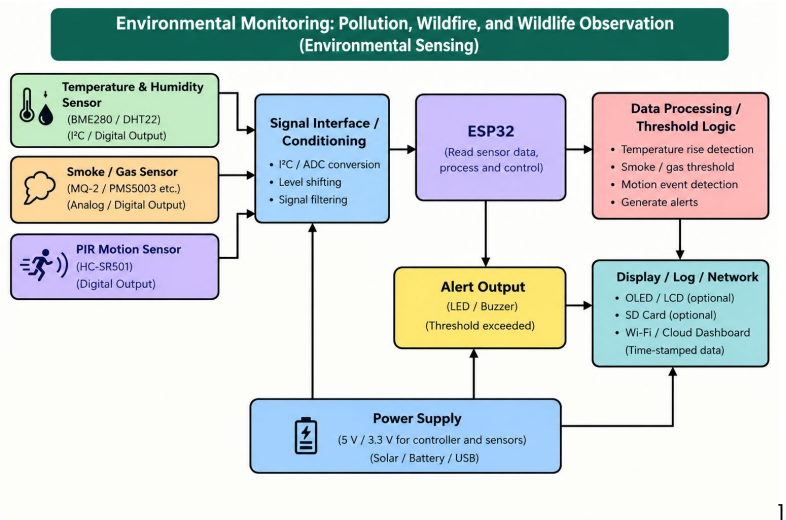
Components Required

ESP32; BME280/DHT22; appropriate smoke/particulate sensor module; PIR or camera trigger; buzzer/LED; weather-resistant enclosure; optional SD card/solar supply.

Interface and Pin Connections

Environmental sensor on I²C or a digital data pin; PIR OUT → GPIO27; status LED → GPIO2 through resistor; buzzer via transistor driver. Keep outdoor electronics dry and ventilated as required by the sensor.

Block Diagram



]

Working Principle

Environmental signals can indicate changing conditions, but fire detection needs sensor fusion, robust thresholds, communications, and false-alarm handling. PIR detects changes in infrared patterns, not species identity.

Implementation Procedure

1.Test temperature/humidity readings indoors first.
2.Establish baseline values and document normal day/night variation.
3.Set conservative prototype alert thresholds and test with safe stimuli only.
4.Verify data logging and communication failure behavior.
5.For wildlife observation, use privacy-respecting placement and avoid disturbing animals.

Program Code / Integration Template

The following sketch is a starting point. Adjust pins, thresholds, sensor scaling, library versions, and driver polarity for the actual hardware before use.

```
// ESP32: DHT22 + PIR event monitor
#include <DHT.h>
#define DHTPIN 4
#define DHTTYPE DHT22
DHT dht(DHTPIN,DHTTYPE);
const int PIR=27, ALERT_LED=2;
const float TEMP_ALERT=40.0; // demonstration threshold only
void setup() {
  Serial.begin(115200); dht.begin();
  pinMode(PIR,INPUT); pinMode(ALERT_LED,OUTPUT);
}
void loop() {
  float t=dht.readTemperature(), h=dht.readHumidity();
  bool valid=!(isnan(t)||isnan(h));
  bool motion=digitalRead(PIR);
  bool alert=motion || (valid && t>=TEMP_ALERT);
  digitalWrite(ALERT_LED,alert?HIGH:LOW);
```

```
  if(valid) Serial.printf("temp=%.1fC humidity=%.1f%% ",t,h);
  else Serial.print("DHT_ERROR ");
  Serial.printf("motion=%d alert=%d\n",motion,alert);
  delay(2000);
}
```

Proposed Test Plan

Test condition	Expected observation
Normal indoor conditions	Values are logged without unexpected alert.
Safe warm-air stimulus	Temperature trend changes gradually.
Motion across PIR field	Event is registered.
Sensor/network loss	Failure is visibly indicated.

Expected Results

The node should report environmental readings and event triggers. It must not be described as proven wildfire prediction or reliable wildlife identification.

Limitations, Safety, and Future Scope

Outdoor weather, condensation, smoke composition, sensor drift, and false triggers affect performance. Never test fire or smoke alarms using uncontrolled flames or hazardous smoke.

Future scope: Add calibrated data logging, a dashboard, fault notifications, enclosure/power design, and a documented evaluation dataset. Any performance claim should be supported by repeatable measurements.